

Tabla de contenido

Módulo 5 – Construir.....	2
Clase 6. Fundamentos de no-code automation y diseño de workflows.....	2
Resultados de aprendizaje observables.....	2
Parte 1.....	2
No-code automation.....	2
Proceso, tarea y workflow.....	3
Triggers y eventos.....	4
Actions, conditions y data mapping.....	4
Parte 2.....	5
Herramientas de automatización.....	5
Creación conceptual de un flujo automatizado.....	5
Cuándo no automatizar.....	5
Casos de uso pertinentes.....	6
Limitaciones, riesgos, controversias y errores frecuentes.....	6
Buenas prácticas.....	6
Demostración sugerida para el docente.....	6
Preguntas de reflexión o comprobación.....	6
Ideas clave para convertir en diapositivas.....	6
Fuentes utilizadas en esta clase.....	7
Bibliografía.....	8

Módulo 5 – Construir

Clase 6. Fundamentos de no-code automation y diseño de workflows

Propósito. Comprender los componentes de una no-code automation y decidir qué procesos son adecuados antes de configurarlos.

Resultados de aprendizaje observables

- Distinguir tarea, proceso, workflow y automatización.
- Identificar triggers, acciones, datos, condiciones y excepciones.
- Representar un flujo de extremo a extremo con responsables y estados.
- Reconocer cuándo no automatizar.

Pregunta orientadora: ¿Qué condiciones hacen que un proceso sea automatizable sin volverlo frágil o riesgoso?

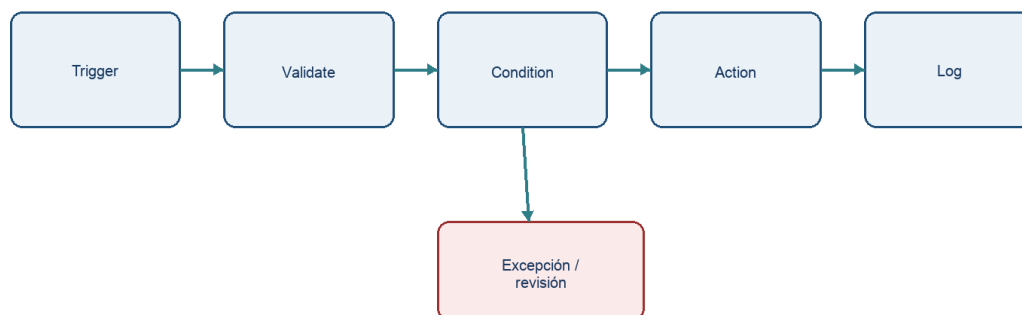


Figura 9. Estructura general de un workflow. Elaboración propia con base en las fuentes citadas en la sección. Texto alternativo: Diagrama conceptual con bloques y flechas que resume la relación central de la clase.

Parte 1

No-code automation

Explicación intuitiva. No-code automation permite configurar integraciones y reglas mediante interfaces visuales. “No-code” describe la experiencia de construcción; no significa ausencia de lógica, software, APIs, pruebas, seguridad o mantenimiento. El objetivo es ejecutar tareas repetibles de forma consistente.

Un flujo simple responde: cuando ocurre X, valida Y, realiza Z y registra el resultado. AI puede ser un paso probabilístico dentro del workflow, no necesariamente el controlador completo.

Explicación técnica. Las plataformas conectan servicios mediante API, webhooks o polling, transforman payloads, almacenan credentials y ejecutan steps. Las abstracciones ocultan código, pero siguen dependiendo de contratos, límites, errores y versiones externas [26], [27].

Un componente LLM debe tener input delimitado, output schema, timeout, cost limit y ruta de exception. Structured validation es necesaria antes de usar el output en una action.

Cuándo conviene y cuándo no. Conviene para procesos estables, frecuentes, basados en reglas y con sistemas integrables. No conviene cuando las excepciones dominan, el proceso cambia continuamente o nadie es dueño.

Antes de automatizar conviene estandarizar y medir el proceso.

Ejemplo correcto. Registrar una solicitud validada, clasificarla dentro de categorías permitidas, escalar baja confianza y notificar al responsable.

Contraejemplo o error frecuente. Automatizar un proceso que el equipo no puede describir o dar al LLM texto libre para elegir cualquier destinatario.

Qué puede simplificar el docente. Puede usarse trigger → validate → decide → act → log. Debe conservarse que no-code no elimina ingeniería ni operación.

Proceso, tarea y workflow

Explicación intuitiva. Una tarea es una unidad de trabajo; un proceso coordina tareas, personas y sistemas para lograr un resultado; un workflow representa la secuencia, estados, decisiones y handoffs. Automatizar una tarea no equivale a mejorar el proceso completo.

El “happy path” muestra cuando todo sale bien. Un diseño real también incluye datos ausentes, duplicados, rechazo, espera, corrección y salida manual.

Explicación técnica. Un workflow puede modelarse como un grafo de nodos y transiciones con inputs, outputs, conditions, state y responsables. Debe definir trigger, terminal states y system of record. Handoffs cambian ownership y son fuentes frecuentes de error.

Procesos probabilísticos requieren thresholds y exception paths. El workflow determinista debe contener al modelo: valida su output, limita actions y registra decisiones.

Cuándo conviene y cuándo no. Conviene mapear as-is y to-be, incluido volumen y exceptions. No conviene ocultar excepciones para simplificar el diagrama ni optimizar un paso aislado que crea retrabajo downstream.

Un system of record claro evita que diferentes aplicaciones mantengan estados contradictorios.

Ejemplo correcto. El flujo de presupuesto incluye solicitud, validación, revisión, aprobación, registro y ruta cuando falta un centro de costo.

Contraejemplo o error frecuente. Mostrar solo formulario → AI → correo y omitir aprobación, error, duplicado y registro.

Qué puede simplificar el docente. Puede usarse ruta principal + desvíos + salida segura. Deben conservarse estado, ownership y system of record.

Triggers y eventos

Explicación intuitiva. Un trigger es la condición o el evento que inicia un workflow: nuevo formulario, cambio de estado, horario o webhook. Es el timbre del sistema, pero puede sonar dos veces. Elegirlo mal produce duplicados, ejecución temprana o flujos que nunca arrancan.

Evento no es lo mismo que condición de negocio. “Fila actualizada” puede ocurrir por muchos cambios; “solicitud pasó a aprobada” expresa el estado relevante.

Explicación técnica. Webhooks envían eventos; polling consulta periódicamente. Los sistemas pueden ofrecer at-least-once delivery, por lo que el consumidor debe tolerar duplicados. Cada evento necesita ID, timestamp, source, type y payload mínimo.

Debounce, filters y idempotency keys controlan repeticiones. La secuencia de eventos puede llegar fuera de orden; state checks evitan aplicar una transición obsoleta.

Cuándo conviene y cuándo no. Conviene un evento inequívoco y con identificador. No conviene disparar procesos sensibles con “cualquier correo nuevo” o texto no autenticado.

Los reintentos deben probarse desde el diseño.

Ejemplo correcto. Iniciar cuando `status` cambia a `approved` y el event ID no ha sido procesado.

Contraejemplo o error frecuente. Cada edición de una fila crea otra factura porque el trigger es “row updated”.

Qué puede simplificar el docente. Puede decirse “el evento que enciende el flujo”. Deben conservarse duplicados, orden, polling/webhook e idempotency.

Actions, conditions y data mapping

Explicación intuitiva. Una action consulta, crea, actualiza o notifica. Una condition decide qué ruta seguir. Data mapping conecta campos entre aplicaciones. Cada bloque debe declarar qué recibe, qué valida y qué entrega.

El texto “quinientos mil” no es necesariamente el número 500 000; una fecha puede cambiar de formato; un campo vacío puede convertirse en cero. Muchos fallos de automation son problemas de datos, no de AI.

Explicación técnica. Los payloads tienen types, schemas, required fields y encodings. Se necesitan validation, normalization, null handling y allowlists. Actions pueden no ser atómicas: una falla intermedia deja estado parcial.

Tool calls generadas por LLM deben validarse contra JSON Schema, permissions y business rules. El modelo no debe construir libremente URLs, destinatarios o montos sin restricciones.

Cuándo conviene y cuándo no. Conviene validar antes de escribir y registrar IDs. No conviene usar texto generado como comando directo ni asumir que aplicaciones comparten formatos.

Para acciones irreversibles se requiere approval o compensating action.

Ejemplo correcto. Validar que monto sea numérico, moneda permitida y destinatario pertenezca a una allowlist antes de enviar.

Contraejemplo o error frecuente. Enviar un correo a la dirección que el modelo extrajo sin comprobar dominio, formato ni autorización.

Qué puede simplificar el docente. Puede usarse recibe → valida → transforma → entrega. Deben conservarse schemas, estado parcial y permisos.

Parte 2

Herramientas de automatización

Explicación intuitiva. Zapier, Power Automate, Make y n8n comparten bloques de triggers, acciones y lógica, pero difieren en conectores, despliegue, gobernanza, costos y extensibilidad. Las funciones exactas dependen de versión y plan [26], [27].

Explicación técnica. Técnicamente, conviene comparar autenticación, secretos, historial, reintentos, entornos, exportación, límites, residencia de datos y capacidad de pruebas. La interfaz visual no elimina dependencia de API externas.

Cuándo conviene y cuándo no. Conviene escoger la plataforma compatible con el ecosistema y gobierno de la organización. No conviene decidir solo por cantidad de conectores o una plantilla.

Ejemplo, contraejemplo y simplificación docente. Correcto: comparar requisitos obligatorios. Error: usar una cuenta personal para un proceso institucional. Simplificación docente: conectores + lógica + operación.

Creación conceptual de un flujo automatizado

Explicación intuitiva. Diseñar empieza por evento, resultado, sistema de registro y responsable. Después se agregan validaciones, acciones, excepciones, aprobaciones y notificaciones. AI puede resumir o clasificar dentro del flujo, no controlar necesariamente toda la secuencia.

Explicación técnica. Técnicamente, debe definirse contrato de datos, identificadores, estados y compensaciones. Las salidas generativas se validan por esquema y, si afectan decisiones, por contenido.

Cuándo conviene y cuándo no. Conviene insertar AI en un paso delimitado. No conviene permitir que texto libre seleccione cualquier acción o destinatario.

Ejemplo, contraejemplo y simplificación docente. Correcto: AI propone categoría; regla valida lista y humano revisa baja confianza. Error: AI decide y envía pago. Simplificación docente: AI como nodo probabilístico dentro de un sistema determinista.

Cuándo no automatizar

Explicación intuitiva. No automatizar es una decisión válida cuando el volumen es bajo, el proceso cambia, el impacto del error es alto, faltan permisos o la excepción es la norma. A veces estandarizar primero produce más valor.

Explicación técnica. Técnicamente, el costo total incluye diseño, licencias, monitoreo, incidentes, mantenimiento y dependencia. El retorno debe descontar errores y trabajo de supervisión.

Cuándo conviene y cuándo no. Conviene posponer hasta estabilizar criterios. No conviene automatizar para evitar una decisión organizacional incómoda.

Ejemplo, contraejemplo y simplificación docente. Correcto: documentar y medir antes de construir. Error: automatizar aprobaciones ambiguas. Simplificación docente: repetible, estable, medible y recuperable.

Casos de uso pertinentes

- Registrar solicitudes validadas y notificar responsables.
- Clasificar entradas con AI y escalar baja confianza.
- Generar un borrador de informe desde datos no sensibles.

Limitaciones, riesgos, controversias y errores frecuentes

- Duplicados por reintentos o triggers ambiguos.
- Credenciales compartidas y permisos amplios.
- Estado parcial después de una falla.
- Costos y límites ocultos por plan.

Buenas prácticas

- Mapear camino feliz, excepciones y reversión.
- Usar identificadores únicos e idempotencia.
- Separar ambientes y datos de prueba.
- Mantener un responsable operativo.

Demostración sugerida para el docente

Representar un flujo seguro de recepción de solicitud pública: trigger, validación, clasificación por AI, umbral, revisión y registro. Simular una falla y mostrar la ruta de excepción.

Preguntas de reflexión o comprobación

- ¿Cuál es el sistema de registro?
- ¿Puede repetirse el trigger?
- ¿Qué acción es irreversible?
- ¿Qué parte es probabilística?

Ideas clave para convertir en diapositivas

- No-code no significa sin ingeniería.
- Trigger → validar → decidir → actuar → registrar.
- Camino feliz y excepciones.
- AI como nodo acotado.

- Criterios para no automatizar.

Fuentes utilizadas en esta clase

Referencias principales: [26], [27]. Las características de productos citadas deben verificarse de nuevo antes de impartir la sesión, porque pueden cambiar con versiones, planes y región.

Bibliografía

La bibliografía completa se encuentra en el siguiente archivo:

<https://evinracher.github.io/curso-sinergia/recursos/fundamentos-teoricos.pdf>